

Typed Directed Reachability Graphs for AI-Assisted Security Auditing

A Practical Middle Ground Between Ad Hoc Review and Full Formal Verification

Draft note

7 April 2026

Abstract

This note proposes a practical audit framework based on a typed directed reachability graph whose nodes represent attacker entry points, internal functions, invariants, and optional path-validity conditions. The core idea is to frame vulnerability discovery as the search for paths from attacker-controlled entry points to broken invariants, followed by a second pass that evaluates whether the conditions required to turn such paths into real attacks hold, fail, or remain undetermined. The approach is intended as a middle ground between purely manual review, static pattern matching, symbolic execution, and full formal verification. It is especially attractive for domains in which exploitability depends on composition, intent, and protocol context, including modular smart contracts and zero-knowledge backend implementations. We argue that the graph artifact also improves attribution and quantification of human versus machine contribution by externalizing the audit state into explicit invariants, paths, conditions, and verdicts.

1 Glossary

Substrate. A representation layer that defines the core objects, relations, and transformations of the auditing process. In GRIFT, the substrate is a typed graph over functions, invariants, paths, and conditions.

Invariant. A property that must hold for all valid executions of the system. Invariants may be state-based (for example, balance conservation), protocol-level (for example, account scoping), or proof-level (for example, soundness of a proof system).

Path. A sequence of function executions or semantically relevant transitions representing a potential execution trace from an attacker-controlled surface to a security obligation.

Condition. A predicate that must hold for a path to be exploitable. Conditions capture attacker capabilities, environmental assumptions, observability, or protocol-specific constraints.

Violation. An execution path that breaks an invariant under a set of satisfied conditions.

Audit Order. The number of distinct attacker-controlled entry points or externally usable surfaces required to construct an exploit.

Finding. A human-readable projection of a violation, typically reported as an issue or vulnerability.

Reachability. The existence of a valid execution path in the program or protocol model.

Exploitability. The existence of a reachable path that also satisfies all required conditions for breaking an invariant.

2 Related Work

2.1 Attack Trees and Attack Graphs

Classical security modeling uses attack trees and attack graphs to represent adversarial goals, preconditions, and paths to compromise. Attack trees structure a security objective into subgoals, while attack graphs generalize this idea to reachability over hosts, privileges, and exploit preconditions [1, 2]. This line of work established the general idea that security analysis can be organized as explicit graph search rather than as an unstructured checklist.

2.2 Program Analysis and Symbolic Execution

Program analysis contributes the graph abstractions most directly related to GRIFT’s code-facing layer: control-flow graphs, call graphs, data-flow graphs, and program dependence graphs. Symbolic execution [3] and its descendants explore feasible paths and generate concrete counterexamples, but often suffer from path explosion. In smart-contract security, this tradition appears in tools such as Oyente [5], Securify [6], and Slither [7], which all rely on structured reasoning over program semantics or dependency information.

2.3 Invariant-Based Testing and Invariant Inference

A separate line of work centers the audit process around invariants or derived specifications. Echidna is the most visible smart-contract example: the user writes properties and the tool searches for counterexample call sequences. More recent work studies how invariants can be inferred rather than written manually. Trace2Inv studies invariant effectiveness for smart contracts and presents a specialized inference tool for transaction traces [9]. FLAMES uses a domain-adapted LLM to synthesize Solidity invariants and guards automatically [10]. These works are highly relevant because they show that invariant-centric analysis and even AI-generated invariants are already established ideas.

2.4 Stateful Sequence Search

Many vulnerabilities require multiple calls or carefully ordered state transitions rather than a single bad function. SmarTest addresses this problem by guiding symbolic execution toward vulnerable transaction sequences with a language model [8]. Related work such as reinforcement-learning-based sequence generation similarly explores stateful exploit search rather than isolated detector hits. This literature is especially relevant to GRIFT’s notion of higher-order audit composition.

2.5 Fuzzing and Symbolic Testing

Modern smart-contract auditing also relies heavily on fuzzing and symbolic testing. Echidna and Medusa search over function-call sequences and random or guided inputs; Halmos and related symbolic testing tools reuse property-like specifications or tests as symbolic obligations. These systems are important because they provide practical falsification engines, even when they do not externalize the audit state in graph form.

2.6 Static Analyzers and Detector-Centric Pipelines

Another major class consists of detector-centric tools that search for known vulnerability patterns using static analysis, intermediate representations, dataflow reasoning, and heuristics. In the smart-contract ecosystem, Oyente, Securify, Slither, and Aderyn are representative examples. Their output is typically a flat or ranked list of issues rather than an explicit graph over invariants, paths, conditions, and adjudications.

2.7 LLM-Based and Agentic Auditing

Recent systems extend this trend by using large language models and agentic pipelines to summarize code, hypothesize exploits, retrieve prior knowledge, and triage candidate issues. Public examples include Wake Arena, LLM-SmartAudit, iAudit, and similar systems that combine detectors, retrieval, and LLM issue synthesis. This class is the most relevant comparison point for GRIFT because it also targets AI-assisted security review, but it usually presents the workflow primarily as issue generation and refinement rather than as an explicit audit-state representation.

2.8 Zero-Knowledge (ZK) Auditing Tools

Zero-knowledge systems introduce security obligations that are only partially addressed by current auditing tooling. Existing workflows are largely manual and expert-driven, relying on cryptographic reasoning about soundness, completeness, zero knowledge, transcript binding, Fiat–Shamir domain separation, setup integrity, batching correctness, and circuit or verifier consistency. Some tools cover fragments of this space: circuit analyzers can detect under-constrained signals or malformed constraint systems, while general-purpose analyzers can inspect the surrounding smart contracts and integration code. Picus is a notable formal-methods tool in this direction because it checks uniqueness properties related to under-constrained signals in ZKP circuits [14, 15]. Nevertheless, there is still no widely adopted audit substrate that combines explicit ZK proof obligations, code reachability, exploit composition, and path-validity conditions in a single workflow.

2.9 Formal Verification and Specification Systems

Formal verification and specification tools occupy a different point in the design space. Certora Prover checks smart contracts against rules written in the Certora Verification Language [11]. Scribble is a specification language and runtime verification tool that translates annotations into executable assertions [12]. Runtime Verification’s tooling and similar systems provide specification-driven or symbolic assurance for smart contracts [13]. In the ZK setting, Veridise’s Picus checks uniqueness properties related to under-constrained signals in ZKP circuits [14, 15]. These tools are essential related work because they show what it means to make correctness obligations explicit and machine-checkable, but they generally require stronger specifications and a heavier formal-methods workflow than typical audits.

2.10 Assurance Cases and Structured Arguments

Outside blockchain tooling, assurance-case methods such as Goal Structuring Notation (GSN) organize claims, subclaims, and evidence into explicit argument structures [16]. They are relevant because they emphasize inspectable intermediate artifacts rather than opaque end-to-end reasoning. This is conceptually close to the audit-state externalization that GRIFT aims for, even though GRIFT is not itself a full assurance-case methodology.

2.11 Provenance and Human–AI Attribution

Finally, there is a broader literature on workflow provenance and human–AI responsibility attribution. Provenance systems track how outputs are derived from intermediate transformations, while recent work on human–AI collaboration studies causal and responsibility-aware attribution in mixed systems [17, 18]. This literature matters because one of GRIFT’s goals is to make the contributions of humans and machines attachable to concrete intermediate artifacts rather than only to final issue reports.

2.12 Deterministic Core vs Heuristic Guidance.

GRIFT is fundamentally independent of AI-based techniques. Its core functionality reduces to well-understood computational problems, including graph traversal, path enumeration, and constraint evaluation. As such, it can be implemented using classical algorithms such as breadth-first search, depth-first search, or heuristic-guided search (e.g., A*). AI techniques can be integrated to improve efficiency by guiding search, suggesting invariants, or prioritizing promising paths. However, they are not required for correctness. This separation provides a key advantage over LLM-based auditing tools, where reasoning is implicit and difficult to inspect or refine. In contrast, GRIFT externalizes the reasoning process, enabling explicit representation, validation, and incremental improvement of audit results.

2.13 Positioning of GRIFT

Core Claim. GRIFT is not a detector, fuzzer, or verifier. It is a substrate that represents the audit state as a typed graph over functions, invariants, paths, and conditions, enabling explicit reasoning about exploit composition and separating reachability from exploitability.

Novelty. GRIFT does not introduce new primitives such as graphs, invariants, or symbolic execution. Its contribution is the integration of these elements into a unified audit-state representation that (i) treats invariants as first-class targets of reachability, (ii) separates reachability from exploitability via explicit path-validity conditions, (iii) supports higher-order exploit composition across multiple attacker entry points, and (iv) naturally extends to proof-obligation-style invariants arising in zero-knowledge systems. This combination is not captured by existing smart-contract, fuzzing, symbolic-execution, or LLM-based auditing tools.

Feature Mapping. The most useful way to position GRIFT against prior work is feature-by-feature rather than tool-by-tool.

- **Typed graph substrate.** Like attack graphs and program-analysis graphs, GRIFT relies on explicit graph structure. Unlike them, it combines code-level nodes with security-level nodes in one representation.

- **Invariant-centric reasoning.** Like Echidna, Trace2Inv, and FLAMES, GRIFT treats invariants as first-class objects. Unlike those systems, it embeds invariants inside a larger graph of reachability, conditions, and exploit composition.
- **Reachability versus exploitability.** Like symbolic execution and fuzzing, GRIFT searches candidate paths. Unlike them, it explicitly inserts a second pass over path-validity conditions to separate structural reachability from semantic exploitability.
- **Higher-order composition.** Like sequence-search systems such as SmarTest, GRIFT is concerned with nontrivial multi-step exploits. Its order- k abstraction, however, counts distinct attacker-usable entry surfaces rather than merely the number of transactions in a sequence.
- **Compatibility with existing AI tooling.** Like agentic auditors, GRIFT benefits from LLM assistance for proposing invariants, pruning paths, and extracting conditions. Unlike a pure issue-synthesis loop, it externalizes the audit state into reviewable artifacts.
- **Relation to formal methods.** Like formal verification tools, GRIFT takes specifications and obligations seriously. Unlike full verification systems, it is designed as a practical middle layer between ad hoc review and theorem-heavy proof workflows.
- **Provenance.** Like provenance and human–AI attribution frameworks, GRIFT attaches metadata to intermediate artifacts. In the audit setting, this means invariants, paths, conditions, and verdicts can carry authorship and review history.

3 Deterministic Core and Relation to Classical Methods

Deterministic Core. GRIFT is fundamentally independent of AI-based techniques. Its core functionality reduces to well-understood computational problems, including graph construction, path exploration, and constraint evaluation. Given a program and a set of invariants, GRIFT constructs a typed graph and searches for execution paths that violate these invariants under specified conditions.

Formally, the task can be expressed as finding a path π such that:

$$I(\text{Exec}(\pi)) = 0 \wedge C(\pi)$$

where I is an invariant and $C(\pi)$ encodes path-validity conditions.

This problem can be addressed using classical algorithms such as:

- breadth-first or depth-first search for path exploration,
- shortest-path or heuristic search (e.g., Dijkstra, A^*) for prioritizing candidate executions,
- constraint solving techniques for validating conditions.

Relation to Symbolic Execution. GRIFT shares similarities with symbolic execution in that both explore execution paths. However, symbolic execution typically operates at the level of program semantics and path constraints, whereas GRIFT elevates the analysis to invariant-centered reasoning. In GRIFT, paths are not explored exhaustively but are evaluated relative to their ability to violate explicitly modeled invariants.

Relation to Model Checking. Model checking verifies that a system satisfies a set of properties by systematically exploring its state space. GRIFT can be viewed as a specialized form of model checking where:

- invariants correspond to safety properties,
- execution paths correspond to traces,
- violations correspond to counterexamples.

Unlike traditional model checking, GRIFT explicitly incorporates adversarial conditions and focuses on exploit construction rather than full system verification.

Relation to Attack Graphs. Attack graphs model how an adversary can transition between system states to achieve a goal. GRIFT extends this paradigm by:

- integrating program-level structure (functions and execution),
- treating invariants as explicit attack targets,
- encoding conditions required for exploit feasibility.

Heuristic Role of AI. While GRIFT is executable using deterministic algorithms, practical deployments face challenges such as path explosion and invariant discovery. AI-based techniques can be integrated as heuristic components to:

- prioritize promising paths,
- suggest candidate invariants,
- infer likely attack conditions.

Importantly, AI is not required for correctness. It serves as a guidance mechanism that improves efficiency and usability.

Explainability and Debuggability. A key advantage of this design is that GRIFT externalizes the reasoning process. All intermediate artifacts—paths, invariants, and conditions—are explicitly represented and can be inspected, validated, and refined.

This contrasts with LLM-based auditing systems, where reasoning is implicit in model parameters and outputs are difficult to analyze or systematically improve.

Summary. GRIFT can be understood as a deterministic reasoning substrate grounded in classical algorithms, augmented by optional heuristic guidance from AI. This separation enables both theoretical rigor and practical scalability.

4 Why Another Framework?

Formal verification provides the strongest guarantees but remains expensive, specification-heavy, and difficult to scale across heterogeneous codebases. At the other extreme, purely prompt-based AI review is easy to use but can be difficult to validate, reproduce, and attribute.

Thesis. Modern auditing is increasingly assisted by automated tools that enumerate potential issues. However, critical vulnerabilities often emerge from the composition of multiple conditions and assumptions. We argue that auditing should be modeled as a structured search over invariant-violating executions rather than as the generation of flat issue lists.

Contributions. This note introduces:

- a typed graph substrate for representing audit state;
- an explicit separation between reachability and exploitability;
- an audit-order abstraction for higher-order exploit composition; and
- a framework applicable to both modular smart contracts and zero-knowledge systems.

The proposed framework aims for a practical middle point:

- use explicit invariants rather than informal “bug classes” as the root object of the audit;
- search concrete program structure rather than free-form ideas;
- separate reachability from exploitability;
- make human and machine actions auditable and countable.

5 GRIFT Model

Overview. We model the audit process as a typed directed graph $G = (V, E, \tau)$, where nodes represent code-level and security-level entities, and edges represent reachability, dependency, and violation relationships. The graph is not merely a control-flow or dataflow artifact: it is a representation of the evolving audit state.

Node Types. Each node $v \in V$ has a type $\tau(v)$ in:

- **EntryPoint:** externally reachable attacker-usable surfaces;
- **InternalFunction:** internal functions or semantically relevant helper routines;
- **Invariant:** security, confidentiality, authorization, linkage, or cryptographic obligations;
- **Condition:** contingent facts required for a path to become a real attack;
- **Path:** an optional derived artifact representing an enumerated candidate exploit path;
- **Verdict:** an optional derived artifact recording adjudication.

Edge Types. Edges may encode:

- **Reachability** (e.g. `calls`, `flows_to`): one code artifact can reach another during execution;
- **Risk targeting** (e.g. `risk_for`): a function or path may threaten a specific invariant;
- **Dependency** (e.g. `depends_on`): exploitability depends on a contingent condition;
- **Validation** (e.g. `validated_by`): a condition or path has been adjudicated by a human or by a tool.

Execution Semantics. A candidate path is a sequence

$$\pi = (v_0, v_1, \dots, v_k), \quad v_0 \in V_{\text{EP}}, \quad v_k \in V_{\text{INV}},$$

that corresponds to an attacker-relevant execution from an entry point to an invariant-bearing obligation.

Violation Semantics. A path is not yet a vulnerability merely because it is reachable. GRIFT promotes a reachable path to a candidate attack only after evaluating its path-validity conditions. Formally, a violation instance is characterized by

$$I(\text{Exec}(\pi)) = 0 \wedge C(\pi),$$

where I is the target invariant and $C(\pi)$ is the conjunction of the path-validity conditions associated with π .

Audit Order. We define the *order* of an exploit as the number of distinct attacker-controlled entry surfaces involved in constructing the violating execution. This differs from sequence length: a short exploit may still be second-order if it composes two externally available surfaces that are individually benign or only weakly suggestive.

Provenance. GRIFT records authorship and review at the artifact level rather than trying to assign a single percentage to an entire finding. Typical fields include `proposed_by`, `modified_by`, `accepted_by`, and `reviewed_by`. This allows human and machine contributions to be attached to invariants, conditions, paths, and verdicts separately.

6 Toward a Formalized Audit Process

We do not claim to formalize security auditing in the sense of end-to-end theorem proving. Rather, the claim is more modest and, in our view, more practical: the graph framework is a step toward a *formalized audit process model*. It makes explicit a set of objects that are usually implicit or conversational:

- invariants;
- attacker entry points;
- internal program structure;
- candidate exploit paths;
- path-validity conditions;
- final verdicts.

This explicitness has several consequences. First, it imposes a disciplined decomposition on the audit task. Second, it makes complexity adjustable through audit order. Third, it grounds the search component of the audit in well-understood algorithmic foundations such as graph traversal, shortest-path search, and path pruning. Fourth, it separates structure from heuristics: fuzzing results, detector outputs, known-vulnerability checklists, prior audit reports, and LLM-generated

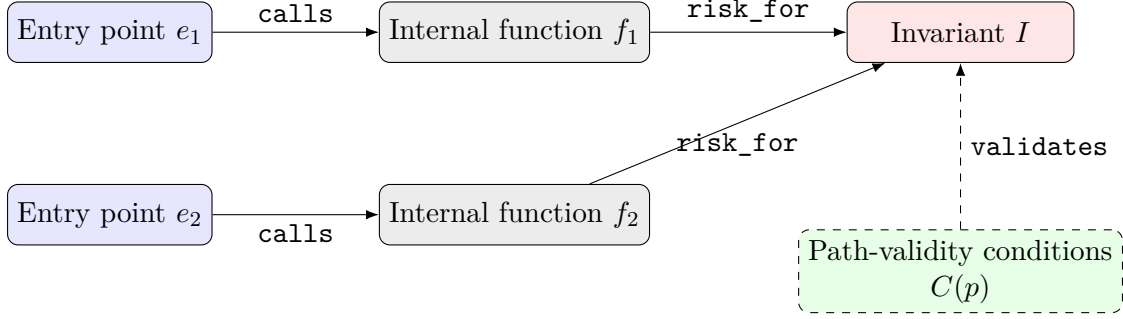


Figure 1: A schematic typed directed reachability graph. Candidate paths run from attacker entry points through internal functions to invariants. Path-validity conditions are attached in a second pass.

hypotheses can all be layered on top of the graph as ranking or pruning signals rather than serving as the primary audit substrate.

In this sense the framework is analogous to foundational moves made in other fields. Shannon’s theory separates information processing from semantic interpretation. Turing’s framework separates mechanizable computation from broader claims about intelligence. In audit work, an analogous separation is useful between:

- systematic search over structured security-relevant possibilities; and
- contextual adjudication of whether a candidate path is actually security-meaningful.

The graph model makes that separation concrete. Machines are well suited to extraction, graph construction, path enumeration, ranking, and consistency checking. Humans remain central in proposing or refining invariants, validating path-validity conditions, and assigning final security meaning. We do not claim that AI contributes only syntactic or mechanical labor; in practice models can also suggest interpretations. The claim is instead that the proposed representation makes such contributions more visible, reviewable, and quantifiable.

7 Typed Directed Reachability Graph

Definition 1 (Audit graph). *Let $G = (V, E)$ be a typed directed graph. The node set is partitioned into*

$$V = V_{\text{EP}} \cup V_{\text{IF}} \cup V_{\text{INV}} \cup V_{\text{COND}},$$

*where V_{EP} are attacker entry points, V_{IF} are internal functions, V_{INV} are invariants, and V_{COND} are optional path-validity conditions. Edges represent relations such as *calls*, *supports*, *risk_for*, *depends_on*, or *validates*.*

In practice, the graph can be built in two phases. A minimal first phase uses only attacker entry points, internal functions, and invariants. A second phase attaches path-validity conditions after candidate paths have been generated. This separation keeps graph construction simple and delegates harder semantic judgments to a later pass.

7.1 Tiny Running Example

Consider a simplified vault with two externally reachable surfaces: `setOracle(address)` and `withdraw(uint256)`. Suppose the intended invariant is:

A user cannot withdraw assets at an attacker-inflated price.

A pure code graph would record that `withdraw` depends on the current oracle price and that `setOracle` changes the oracle address. GRIFT makes the security interpretation explicit:

- entry points: `setOracle`, `withdraw`;
- internal functions: `_setOracle`, `_quote`, `_withdraw`;
- invariant: `INV-price-integrity`;
- candidate paths: `(setOracle, _setOracle, INV)` and `(withdraw, _quote, _withdraw, INV)`.

Neither first-order path alone proves a real issue. The second-order exploit emerges only when they are composed: the attacker first replaces the oracle and then withdraws against the manipulated price. The decisive path-validity conditions are not structural facts but semantic ones, for example that the caller can set the oracle, that the new oracle is attacker-controlled, and that the manipulated price materially influences the withdrawal amount. This is precisely the kind of second-order composition that GRIFT is meant to expose.

7.2 Invariants

The framework intentionally uses a broad notion of invariant. An invariant may be:

- a state safety property (for example, *threshold remains reachable*);
- an authorization property (for example, *only authorized signers can grant ACL rights*);
- a confidentiality property (for example, *signer mapping remains hidden from passive observers*);
- a protocol linkage property (for example, *a disclosed result must be scoped to the validated account*);
- an implementation obligation induced by a cryptographic argument (for example, *Fiat-Shamir challenge derivation binds the full transcript*).

This flexible definition is important. Many practical vulnerabilities do not violate a simple storage invariant; they violate a linkage, binding, or secrecy claim.

7.3 Paths and Attacks

A *candidate attack path* is a path

$$p = (v_0, v_1, \dots, v_k), \quad v_0 \in V_{\text{EP}}, \quad v_k \in V_{\text{INV}},$$

from an attacker entry point to an invariant. The existence of such a path does *not* imply a real vulnerability. It only implies that the invariant is reachable through attacker-controlled program structure.

Each candidate path is paired with a set of path-validity conditions

$$C(p) = \{c_1, \dots, c_m\}.$$

Examples include:

- an account parameter is ignored in a security-critical branch;
- a globally stored fact is reused across distinct accounts;
- a handle is visible in calldata or logs;
- a decryption proof can be obtained once public decryptability is enabled;
- a verifier accepts a challenge derived from an incomplete transcript.

The second pass labels each condition as `holds`, `does_not_hold`, or `undetermined`. Path status is then derived mechanically:

- if any condition is `does_not_hold`, reject the path;
- if all conditions hold, the path is a confirmed candidate and often close to a confirmed issue;
- if no condition is false but at least one is undetermined, retain the path for deeper review.

8 Audit Orders

The framework naturally supports *audit order*. A first-order audit allows exploits initiated from a single attacker entry point. A second-order audit allows exploits that require composition of two attacker-usable surfaces. Higher-order audits generalize this idea.

Definition 2 (Order- k audit). *An order- k audit permits exploit constructions that activate at most k distinct attacker entry points or public surfaces.*

This concept is useful because many meaningful confidentiality breaks are not first-order. For example, one entry point may enumerate hidden handles while another turns adaptive queries into an oracle. Separately they are weak; together they become a concrete exploit. Crucially, audit order counts distinct attacker-usable entry surfaces, not the number of internal branches or candidate paths reachable from a single entry point.

Order-1 search is usually cheap and high-signal. Order-2 search often finds the most interesting composed attacks. Order-3 and higher rapidly increase the search space and should usually be targeted to already suspicious clusters.

9 Algorithms Without AI

In principle, path search over the graph does not require AI. Standard graph algorithms suffice:

- depth-first search (DFS) for full reachability or exhaustive path enumeration in bounded-depth settings;
- breadth-first search (BFS) for shortest unweighted candidate paths;
- Dijkstra’s algorithm [4] or A* when costs are assigned to nodes or edges;
- dominator analysis or SCC decomposition to compress search in large graphs;
- topological dynamic programming when the graph or a projection of it is acyclic.

The challenge is not raw reachability. The challenge is pruning the combinatorial space of *plausible* exploit paths and interpreting whether path-validity conditions are semantically satisfied. This is precisely where AI is valuable.

10 Where AI Helps

AI is most useful in four places:

1. **Invariant proposal.** Models can suggest candidate invariants from code, comments, tests, or protocol documentation.
2. **Path pruning.** Models can rank candidate paths by likely exploitability and group variants, prerequisites, and impact-only paths.
3. **Condition extraction.** Models can translate a raw reachability path into the specific conditions that must hold for a real exploit.
4. **Context integration.** Models can incorporate deployment assumptions, off-chain protocol behavior, library semantics, and intended usage.

This division of labor is important. Classical algorithms can enumerate possibilities, but they do not understand intent. AI can use context to suppress obviously meaningless paths, promote composition hypotheses, and generate concrete exploit narratives. In this sense AI functions as a context-sensitive pruning and interpretation layer, not as a replacement for explicit structure.

11 Applicability to Smart Contracts and ZK Backends

11.1 Smart contracts

For modular smart contracts the framework is a natural fit. Entry points are public or external functions, internal functions come from local code and inherited parents, and invariants include authorization, replay protection, state consistency, and confidentiality claims. The graph can also absorb off-chain facts such as relayer behavior, proof retrieval, and mempool visibility through path-validity conditions.

11.2 ZK backends

The same methodology is promising for ZK backend audits, although the meaning of invariants shifts. Instead of storage invariants, the key properties are soundness, completeness, zero knowledge, transcript binding, domain separation, setup integrity, batching equivalence, subgroup membership, and serialization correctness. The entry points are verifier APIs, proof parsers, setup loaders, batch-verification front doors, transcript constructors, and recursion wrappers.

Fiat-Shamir vulnerabilities are especially well suited to this style of analysis. A representative invariant is:

Every verifier challenge must be derived from the complete and correctly domain-separated transcript.

Candidate paths then start at transcript-construction or verifier entry points and traverse internal helper functions responsible for serialization, label binding, batching metadata, and domain separation. The path-validity conditions capture the true security content: whether a field is omitted, whether labels are attacker-controlled, whether transcript resets or cross-instance collisions are reachable, and whether the resulting challenge reuse creates a soundness failure. This is not full theorem proving, but it is much closer to the theorem obligations than purely syntactic tools.

There are currently no widely adopted, dedicated audit tools for ZK backends that combine theorem-aware invariants, code reachability, composition search, and explicit exploit-condition tracking in one workflow. For that reason, the graph framework looks promising as an organizing principle: it is expressive enough to model cryptographic obligations but lightweight enough to remain practical.

12 Quantifying Human and Machine Contribution

A recurring concern with current AI audit tooling is blurred authorship. If an issue emerges from a long conversation, it is hard to tell whether the machine contributed 10%, 80%, or 95% of the final result. The graph framework can improve this situation because the audit state is decomposed into explicit artifacts:

- invariants;
- node and edge construction;
- candidate paths;
- path-validity conditions;
- condition adjudications;
- final verdicts.

Each artifact can carry provenance:

- `source = human` if the invariant was introduced manually;
- `source = machine` if it was proposed by an assistant;
- `validated_by = human` if a person confirmed the path conditions;
- timestamps or edit logs for who added, deleted, or reclassified a path.

This does not solve attribution perfectly, but it makes it substantially easier to quantify. A report can explicitly state, for example, that the machine proposed 40 candidate paths, the human rejected 7, confirmed 3, rewrote 2 invariants, and supplied the decisive condition adjudication for all confirmed attacks. That is far more measurable than a monolithic AI-generated issue list.

13 Results and Illustrative Case Studies

We do not yet claim a quantitative benchmark. The goal of this section is instead to show that the GRIFT representation is expressive enough to capture qualitatively different classes of security failures: a classic code-level bug, an economic manipulation, a ZK proof-system failure, and a cross-component confidentiality break.

13.1 Code-Level Baseline: Reentrancy

Invariant. The total asset balance must equal the sum of user-withdrawable balances.

Path. `deposit` $\rightarrow$ `withdraw` $\rightarrow$ external callback $\rightarrow$ reentrant `withdraw`.

Path-validity conditions. The victim contract performs an external call before updating internal accounting; the attacker controls the callee and can reenter before state is synchronized.

GRIFT view. This is a first-order path from a single entry surface to a state-safety invariant. It serves as a sanity check that the framework captures standard smart-contract issues without requiring special cryptographic machinery.

13.2 Economic Example: ERC4626 Inflation Attack

Invariant. Share issuance must remain proportional to contributed assets and must not be manipulable through attacker-timed donations.

Path. Empty vault $\rightarrow$ attacker deposit $\rightarrow$ attacker donation $\rightarrow$ victim deposit.

Path-validity conditions. The vault is initially empty or thinly capitalized; rounding rules or exchange-rate initialization allow the donation to skew the effective share price; the attacker can time the victim deposit after the skew.

GRIFT view. This example highlights that many relevant invariants are economic rather than purely syntactic. The exploit is not a low-level code bug but a violation of a protocol-level fairness and proportionality obligation.

13.3 ZK Case Study: The Last Challenge Attack

Background. The Fiat–Shamir transform derives verifier challenges from a transcript. Security requires that each challenge bind all relevant prior commitments and messages [21, 19, 20]. If a challenge omits transcript elements, a malicious prover may regain adaptive control over values that should have been fixed before the challenge is sampled.

Invariant.

Soundness: the verifier must reject proofs of false statements.

This is a proof-level invariant rather than a simple state predicate.

Attack path. The attacker enters through the verifier API, reaches the transcript-construction logic, exploits an omitted dependency in the final challenge, and then returns to the final pairing or polynomial checks with a forged proof:

`verifyProof  $\rightarrow$  appendTranscriptData  $\rightarrow$  deriveFinalChallenge  $\rightarrow$  finalCheck  $\rightarrow$  INV-soundness.`

Path-validity conditions. The final challenge does not bind all commitments; the prover can choose omitted commitments adaptively; and the verifier accepts the resulting proof without independently restoring the missing dependency.

Violation. Under these conditions, a false statement can be accepted. GRIFT treats this as a first-order violation of a proof obligation, but one whose conditions live in transcript semantics rather than in local storage updates.

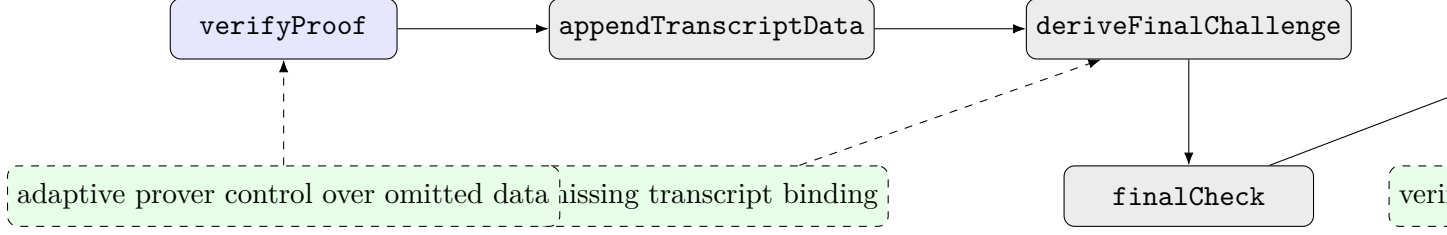


Figure 2: GRIFT view of a Last-Challenge-style soundness break. The main path is short and code-anchored, but the decisive semantics sit in proof-level path-validity conditions concerning transcript binding and adaptive control.

13.4 Cross-Component Confidentiality Case: Confidential Multisig

The following example is adapted from an audit of a confidential multisig system. The goal is to model a confidentiality break in which a publicly decryptable validity bit becomes an oracle over the secret signer set and threshold.

Example graph. Let the relevant nodes be:

- $e_1 = \text{requestDiscloseOperation}$
- $e_2 = \text{getSigners}$
- $f_1 = \text{_validateSignatures}$
- $f_2 = \text{_requestDiscloseOperation}$
- $i_1 = \text{INV20: confidential policy not silently downgraded}$

Edges:

$$e_1 \rightarrow f_1, \quad e_1 \rightarrow f_2, \quad e_2 \rightarrow i_1, \quad f_1 \rightarrow i_1, \quad f_2 \rightarrow i_1.$$

This already identifies three first-order candidate paths:

$$\begin{aligned} P_1 &= (e_1, f_1, i_1), \\ P_2 &= (e_1, f_2, i_1), \\ P_3 &= (e_2, i_1). \end{aligned}$$

Second-order composed attack. The stronger exploit is second-order and composes these surfaces:

1. observe at least one real signature on a chosen hash from public calldata;
2. enumerate encrypted signer handles via `getSigners`;
3. repeatedly call `requestDiscloseOperation` with varied handle selections;
4. obtain public decryption of the resulting validity bit;
5. infer which encrypted handles correspond to real signers and, by repeated tests, learn threshold or weight information.

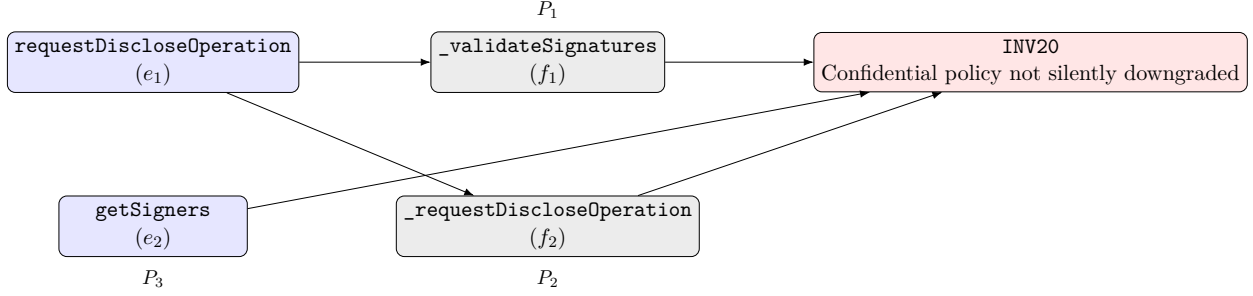


Figure 3: Example graph for a confidentiality-oriented audit of a confidential multisig component. The second-order exploit composes the two entry surfaces e_1 and e_2 ; P_1 and P_2 are two first-order branches reachable from e_1 .

The corresponding path-validity conditions are:

- valid signatures are publicly observable and reusable for the chosen hash;
- encrypted signer handles are enumerable or otherwise visible;
- the validity bit is made publicly decryptable;
- repeated observations can be adaptively chosen;
- the observed bit is sufficiently informative to reconstruct confidential policy.

This example illustrates why a first-order search is necessary but not sufficient. The individual paths are suggestive, but the real exploit emerges only when multiple attacker-usable surfaces and observability conditions are composed. Under the audit-order convention used here, the example remains second-order because it combines two entry surfaces, e_1 and e_2 , even though multiple first-order branches are explored from e_1 .

14 Conclusion

Typed directed reachability graphs offer a practical audit abstraction for AI-assisted security review. They preserve explicit structure, support classical graph search, admit higher-order exploit composition, and create a clean place to encode the conditions that turn mere reachability into real attacks. The framework appears especially promising for domains in which vulnerabilities depend on context and composition, such as modular smart contracts and ZK backend implementations. Most importantly, it improves inspectability and provenance: the machine can help search and prune, but the graph makes clear which invariants, paths, conditions, and verdicts were contributed or validated by humans.

References

- [1] Bruce Schneier. Attack Trees. *Dr. Dobbs's Journal*, 24(12):21–29, 1999.
- [2] Cynthia Phillips and Laura Painton Swiler. A Graph-Based System for Network-Vulnerability Analysis. In *Proceedings of the 1998 Workshop on New Security Paradigms*, pages 71–79, 1998.

- [3] James C. King. Symbolic Execution and Program Testing. *Communications of the ACM*, 19(7):385–394, 1976.
- [4] Edsger W. Dijkstra. A Note on Two Problems in Connexion with Graphs. *Numerische Mathematik*, 1:269–271, 1959.
- [5] Loi Luu, Duc-Hiep Chu, Hrishi Olickel, Prateek Saxena, and Aquinas Hobor. Making Smart Contracts Smarter. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*, pages 254–269, 2016.
- [6] Petar Tsankov, Andrei Dan, Dana Drachsler-Cohen, Arthur Gervais, Florian Buenzli, and Martin Vechev. Securify: Practical Security Analysis of Smart Contracts. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, pages 67–82, 2018.
- [7] Josselin Feist, Gustavo Grieco, and Alex Groce. Slither: A Static Analysis Framework for Smart Contracts. In *Proceedings of the 2nd International Workshop on Emerging Trends in Software Engineering for Blockchain*, pages 8–15, 2019.
- [8] Sunbeom So, Seongjoon Hong, and Hakjoo Oh. SmarTest: Effectively Hunting Vulnerable Transaction Sequences in Smart Contracts through Language Model-Guided Symbolic Execution. In *USENIX Security Symposium*, pages 1361–1378, 2021.
- [9] Zijian Chen, Shengan Zheng, Yuhong Nan, and others. Demystifying Invariant Effectiveness for Securing Smart Contracts. In *Proceedings of the ACM on Software Engineering*, 2024.
- [10] Authors omitted for draft anonymity. FLAMES: Domain-adapted LLM Synthesis of Solidity Invariants and Guards. *arXiv preprint arXiv:2510.21401*, 2025.
- [11] Certora. Certora Prover Documentation. <https://docs.certora.com/>.
- [12] ConsenSys Diligence. Scribble: Specification Language and Runtime Verification Tool for Solidity. <https://docs.scribble.codes/>.
- [13] Runtime Verification. Smart Contract Analysis and Verification. <https://runtimeverification.com/smartcontract>.
- [14] Veridise. Picus: Automated Verification of Uniqueness Properties for ZKP Circuits. <https://github.com/Veridise/Picus>.
- [15] Veridise. Blockchain Security Tools. <https://veridise.com/security/tools/>.
- [16] Tim Kelly and Rob Weaver. The Goal Structuring Notation—A Safety Argument Notation. In *Proceedings of the Dependable Systems and Networks Workshop on Assurance Cases*, 1998.
- [17] Yolanda Gil, Varun Ratnakar, and others. A Survey of Workflow Provenance. *Data & Knowledge Engineering*, 2021.
- [18] Authors omitted for draft anonymity. Causal Responsibility Attribution for Human–AI Systems. *arXiv preprint arXiv:2411.03275*, 2024.
- [19] OpenZeppelin. The Last Challenge Attack. <https://www.openzeppelin.com/news/the-last-challenge-attack>.

- [20] Ariel Gabizon and Zachary J. Williamson. The Last Challenge Attack. *IACR ePrint*, 2024/398, 2024.
- [21] Amos Fiat and Adi Shamir. How to Prove Yourself: Practical Solutions to Identification and Signature Problems. In *Advances in Cryptology—CRYPTO '86*, pages 186–194, 1986.

Class / Representative systems	Primary object	Search / reasoning space	Typical output	Relation to GRIFT
Attack trees / attack graphs	Adversarial goals, exploit preconditions	Goal decomposition, privilege reachability	Attack scenarios	Similar graph intuition; less code-anchored and less centered on invariants plus conditions
Program analysis / symbolic execution (Oyente, Securify, Slither)	Program paths, dependency facts	Feasible executions, semantic facts	Warnings, violations, traces	Similar code-graph reasoning; GRIFT makes invariants and exploit conditions first-class
Invariant testing / inference (Echidna, Trace2Inv, FLAMES)	Properties and inferred invariants	Counterexample search, trace mining, synthesis	Invariants, guards, falsifying sequences	Similar invariant focus; GRIFT embeds invariants in a broader audit-state graph
Stateful sequence search (SmarTest)	Transaction sequences	Stateful multi-call search	Vulnerable sequences	Similar compositional concern; GRIFT models exploit order over attacker surfaces
Fuzzing / symbolic testing (Medusa, Halmos)	Inputs and test properties	Randomized or symbolic exploration	Counterexamples, traces	Similar falsification role; GRIFT is a substrate above these engines
LLM / agentic auditing (Wake Arena, iAudit, LLM-SmartAudit)	Candidate issues and explanations	Issue generation, ranking, retrieval	Finding lists, triage narratives	Similar AI assistance; GRIFT centers explicit intermediate artifacts instead of findings
Formal verification / specs (Certora, Scribble, Picus)	Specifications and proof obligations	Exhaustive proof or runtime checking	Proofs, assertion violations, verified obligations	Similar concern for explicit obligations; GRIFT is lighter-weight and audit-oriented
Assurance cases / provenance	Claims, evidence, derivations	Structured argument, workflow derivation	Assurance arguments, provenance logs	Similar emphasis on inspectable artifacts and attribution

Table 1: Positioning GRIFT in the surrounding literature. The table emphasizes primary objects and outputs rather than ranking systems by strength.